

MemGPT：实现作为操作系统的 LLM

查尔斯·帕克¹ 莎拉·伍德斯¹ 凯文·林⁽¹⁾
Vivian Fang¹ Shishir G. Patil¹ Ion Stoica¹ Joseph E. Gonzalez⁽¹⁾

摘要

大型语言模型（LLM）给人工智能带来了革命性的变化，但却受到有限上下文窗口的限制，妨碍了它们在前对话和文档分析等任务中的应用。为了使上下文的使用超越有限的上下文窗口，我们提出了 *虚拟上下文管理技术*，该技术从传统操作系统中的分层内存系统中汲取灵感，通过在物理内存和磁盘之间分页来提供扩展虚拟内存的假象。利用这种技术，我们引入了 MemoryGPT（MemoryGPT）系统，它能智能地管理不同的存储层，以便在 LLM 有限的上下文窗口内有效地提供扩展上下文。我们在两个领域评估了受操作系统启发的设计，在这两个领域中，现代 LLM 的有限上下文窗口严重影响了它们的性能：文档分析，MemGPT 能够分析远远超过 LLM 上下文窗口的大型文档；多会话聊天，MemGPT 可以创建会话代理，通过与用户的长期交互来记忆、反映和动态发展。我们在 <https://research.memgpt.ai> 上发布了 MemGPT 代码和实验数据。

1. 引言

近年来，大型语言模型（LLMs）及其底层转换器架构（Vaswani 等人，等人，Brown 等人，2017 年；Devlin 2018 年；2020 年；Ouyang 等人，2022 年）已成为对话式人工智能的基石，并带来了一系列广泛的消费者和企业应用。尽管取得了这些进步，但 LLM 使用的有限固定长度上下文窗口极大地阻碍了它们在长对话或长文档推理中的应用。例如，最广泛使用的开源

在超过最大输入长度之前，LLMs 只能支持几十个来回对话或推理一个简短的文档（Touvron 等人，2023 年）。

由于变压器架构的自注意机制，直接延长变压器的上下文长度会导致计算时间和内存成本的二次增长，这使得设计新的长上下文架构成为一项紧迫的研究挑战（Dai 等人，2019；Ki-tae 等人，2020；Beltagy 等人，2020）。虽然开发更长的模型是一个活跃的研究领域（Dong 等人，2023 年），但即使我们能克服上下文缩放的计算难题，最近的研究也表明，长上下文模型很难有效地利用额外的上下文（Liu 等人，2023a）。因此，考虑到训练最先进的 LLM 所需的大量资源以及上下文缩放的收益递减，我们亟需替代技术来支持长上下文。

在本文中，我们将研究如何在继续使用固定上下文模式的同时，提供无限上下文的假象。我们的方法借鉴了虚拟内存分页的理念，该理念通过在主内存和磁盘之间分页数据，使应用程序能够处理远远超出可用内存的数据集。我们利用最近在 LLM 代理函数调用能力方面取得的进展（Schick 等人，2023；Liu 等人，2023b），设计了用于 *虚拟上下文管理* 的受操作系统启发的 LLM 系统 MemGPT。通过函数调用，LLM 代理可以读写外部数据源、修改自身上下文并选择何时向用户返回响应。

这些功能允许 LLM 在上下文窗口（类似于操作系统中的“主存储器”）和外部存储器（类似于传统操作系统中的分层存储器）之间有效地“分页”进出信息。此外，还可以利用函数调用来管理上下文管理、响应生成和用户交互之间的控制流。这样，代理就可以选择迭代修改其上下文中的内容，从而更有效地利用其有限的上下文。

在 MemGPT 中，我们将上下文窗口视为受限的内存资源，并为 LLM 设计了类似于传统操作系统中使用的内存层的内存结构（Patterson 等人，1988 年）。传统操作系统中的应用程序

¹加州大学伯克利分校。
Packer。 <cpacker@berkeley.edu>

通讯地址 Charles



图
它建
议

在其
"LL
LLM "
中, "ph
"与 "ph "
流向
"simi
tual" 。
是

结合使用内存层次结构、操作系统功能和基于事件的控制流，
MemGPT 的组成部分。
MemGPT 可以使用具有有限上下文窗口的 LLM 处理无限制上下文。
。为了证明我们受操作系统启发的新 LLM 系统的实用性，我们在
两个领域对 MemGPT 进行了评估，在这两个领域中，现有 LLM 的
性能受到了有限上下文的严重限制：文档分析领域，标准文本文件
的长度很快就会超过现代 LLM 的输入能力；对话代理领域，受限
于有限对话窗口的 LLM 在扩展对话中缺乏上下文意识、角色一致
性和长期记忆。在这两种情况下，MemGPT 都能克服有限上下文的
限制，从而超越现有的基于 LLM 的方法。

2. MemGPT (MemoryGPT)

MemGPT 受操作系统启发的多级内存架构划分了两种主要内存类型
：主内存（类似于主内存/物理内存/RAM）和外部上下文（类似于
磁盘内存/磁盘存储空间）。主上下文包括 LLM 提示标记-主上下
文中的任何内容都被视为在上下文中，LLM 处理器可在推理过程
中访问。外部上下文指的是 LLM 固定上下文窗口之外的任何信息
。这些超出上下文范围的数据

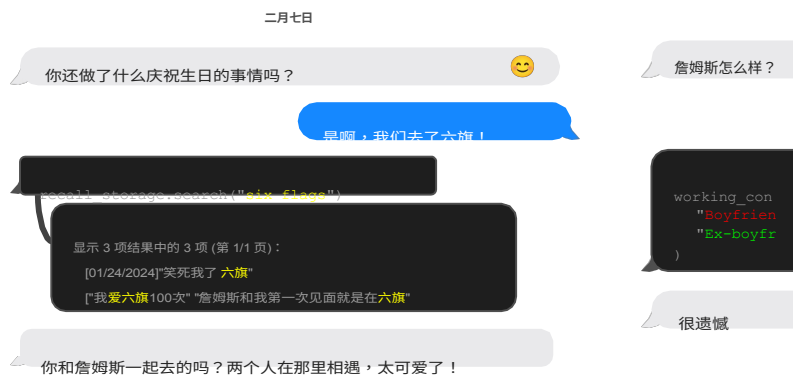


图2.MemGPT（左）可以搜索上下文外的数据，将相关信息引入当前上
下文窗口。

必须始终显式移入主上下文，以便在推理过程中将其传递给 LLM
处理器。MemGPT 提供的函数调用可让 LLM 处理器在无需用户干
预的情况下管理自己的内存。

2.1. 主语境（提示符）

MemGPT 中的提示标记分为三个连续部分：**系统指令**、**工作文本**和
FIFO 队列。系统指令为只读（静态）指令，包含有关 MemGPT 控
制流的信息、不同内存级的预期用途以及如何使用 MemGPT 功能
的指令（例如，如何检索文本外数据）。工作文本是固定大小的非
结构化文本读/写块，只能通过调用 MemGPT 函数写入。在会话设
置中，工作上下文用于存储关键事实、偏好以及其他有关用户和代
理所扮演角色的重要信息，使代理能流畅地与用户对话。先进先出
队列存储的是滚动历史消息，包括代理和用户之间的消息，以及系
统消息（如内存警告）和函数调用输入输出。先进先出队列的第一
个索引存储的是系统消息，其中包含已从队列中驱逐的消息的递归
摘要。

2.2. 队列管理器

队列管理器负责管理召回存储和先进先出队列中的信息。当系统收
到新消息时，队列管理器会将收到的消息添加到先进先出队列中，
连接提示标记，并触发 LLM 推理以生成 LLM 输出（完成标记）。
队列管理器将收到的报文和生成的 LLM 输出写入再调用存储（
MemGPT 报文数据库）。当通过 MemGPT 函数调用检索到调用存
储中的消息时，队列管理器会将它们追加到

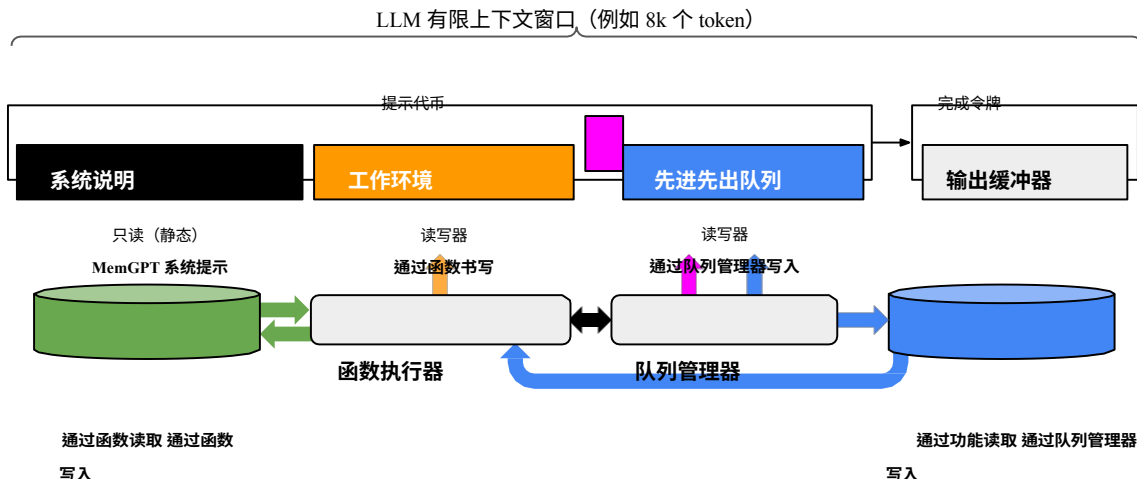


图3在 MemGPT 中，一个固定上下文 LLM 处理器被增强了分层内存系统和功能，使其能够管理自己的内存。LLM 的提示标记（输入）或主上下文由系统指令、工作上下文和 FIFO 队列组成。LLM 的完成标记（输出）被函数执行器解释为函数调用。MemGPT 使用函数在主上下文和外部上下文（存档和调用存储数据库）之间移动数据。LLM 可以通过在其输出中生成一个特殊的关键字参数（request heartbeat=true），请求立即跟进 LLM 推理，以将函数调用链在一起；函数链使 MemGPT 能够执行多步检索，以回答用户查询。

队列，将其重新插入 LLM 的上下文窗口。

队列管理器还负责通过队列驱逐策略控制文本溢出。当提示令牌数超过 LLM 下文窗口的“警告令牌数”（例如上下文窗口的 70%）时，队列管理器会在队列中插入一条系统消息，警告 LLM 即将驱逐队列（“内存压力”警告），以允许 LLM 使用 MemGPT 功能将先进先出队列中的重要信息存储到工作上下文或档案存储（存储任意长度文本对象的读/写数据库）中。当提示标记数超过“刷新标记数”（例如上下文窗口的 100%）时，队列管理器就会刷新队列，以释放上下文窗口的空间：队列管理器会驱逐特定数量的信息（例如上下文窗口的 50%），并使用现有递归摘要和驱逐的信息生成新的递归摘要。一旦队列被刷新，被驱逐的消息就不再处于上下文中，LLM 无法立即查看，但它们会无限期地存储在调用存储器中，并可通过 MemGPT 函数调用进行读取。

2.3. 函数执行器（处理完成令牌）

MemGPT 通过 LLM 处理器生成的函数调用，协调主文本和外部上下文之间的数据移动。内存的编辑和检索完全是自主进行的：MemGPT 可根据当前上下文自主更新和搜索自己的内存。例如，它可以决定何时在上下文之间移动项目（例如，当对话中的“……”出现时）。

如图 1）所示，并修改其主要上下文，以更好地反映其对当前目标和责任的理解（如图 3）。所示我们通过提供明确的指令来实现自导式编辑和再更新，从而指导 LLM 如何与 MemGPT 存储系统进行交互。这些指令包括两个主要部分：(1) 存储器层次结构及其各自实用程序的详细描述；(2) 系统可调用以访问或修改其存储器的函数模式（附带自然语言描述）。

在每个推理周期中，LLM 处理器将主上下文（串联成一个字符串）作为输入，并生成一个输出字符串。MemGPT 会对该输出字符串进行解析，以确保其正确性，如果解析器验证了函数参数，则执行该函数。然后，MemGPT 会将结果反馈给处理器，包括任何运行时出现的错误（例如，在主上下文已达到最大容量时试图向其添加内容）。通过这种反馈回路，系统可以从自己的行为中吸取经验教训，并相应地调整自己的行为。对上下文限制的认识是使自我编辑机制有效工作的一个关键方面，为此，MemGPT 会向处理器发出有关标记限制的警告，以指导其内存管理决策。此外，我们的内存检索机制在设计上也考虑到了这些标记限制，并实施了分页处理，以防止再三调用溢出上下文窗口。

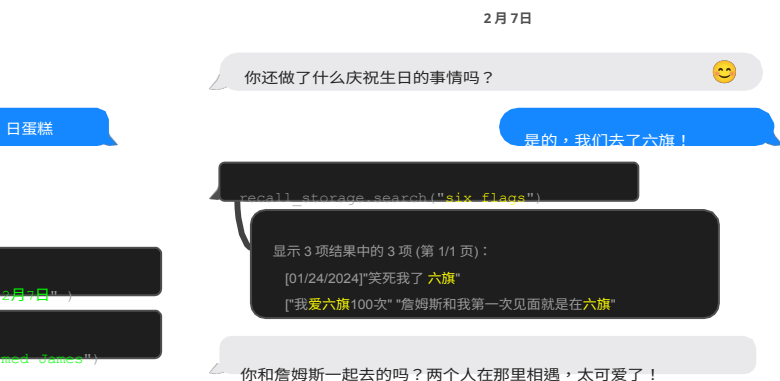


图 4. MemGPT（左）更新存储信息的对话片段示例。此处的信息存储在工作上下文存储器中（位于提示标记内）。

无需用户干预）。MemGPT 使用解析器处理事件，将其转换为纯文本信息，这些信息可附加到主上下文中，并最终作为输入输入到 LLM 处理器中。

许多实际任务需要依次调用多个函数，例如，浏览单个查询结果的多个页面，或整理来自不同文档的数据，这些数据来自不同查询的主要上下文。函数链允许 MemGPT 在将控制权返回给用户之前依次执行多个函数调用。在 MemGPT 中，调用函数时可以使用一个特殊标志，要求在请求的函数执行完毕后立即将控制权返回给处理器。如果该标志存在，MemGPT 将把函数输出添加到主上下文中（而不是暂停处理器执行）。如果该标志不存在（*屈服*），MemGPT 将不运行 LLM 处理器，直到下一个外部事件触发（如用户信息或计划中断）。

3. 实验

我们在两个长语境领域对 MemGPT 进行了评估：对话代理和文档分析。对于会话代理，我们扩展了现有的多会话聊天数据集（Xu 等人，2021 年），并引入了两个新的对话任务，以评估代理保留知识的能力

跨长对话。在文档分析方面，我们在现有任务（Liu 等人，2023a）中对 MemGPT 进行了基准测试，这些任务包括问题解答和冗长文档的键值检索。我们还提出了一个新的嵌套键值检索任务，需要整理多个数据源的信息，这考验了代理整理多个数据源信息（多跳检索）的能力。我们公开发布了我们的增强 MSC 数据集、嵌套键值检索数据集和 2,000 万维基百科文章的 em-bedding 数据集，以促进未来的研究。我们的基准测试代码可在 <https://research.memgpt.ai> 上获取

实施细节。在讨论 OpenAI 模型时，除非另有说明，否则 "GPT-4 Turbo" 指的是特定的 gpt-4-1106-preview 模型端点（上下文窗口为 128,000），"GPT-4" 指的是 gpt-4-0613（上下文窗口为 8,192），而 "GPT-3.5 Turbo" 指的是 gpt-3.5-turbo-1106（上下文窗口为 16,385）。在实验中，我们使用所有基准模型（GPT-4、GPT-4 Turbo 和 GPT 3.5）运行 MemGPT，以显示非基准模型的性能对 MemGPT 的影响。

3.1. 对话式代理的 MemGPT

虚拟伴侣和人性化助手等对话代理旨在与用户进行自然、长期的互动，时间可能长达数周、数月甚至数年。这给具有固定长度上下文的模型带来了挑战，因为它们只能参考有限的对话历史。无限上下文代理应能无缝处理无边界或无重置的连续交流。在与用户对话时，这样的代理必须满足两个关键标准：(1) 一致性--代理应保持对话的一致性。所提及的新事实、偏好和事件应与用户和代理之前的陈述保持一致。(2) 参与性 - 代理应利用关于用户的长期知识来个性化用户。

表 2. 深度记忆检索（DMR）成绩。在这项任务中，会问代理一个关于之前对话（第 1-5 节）中讨论过的话题的具体问题。根据黄金答案对代理的回答进行评分。MemGPT 的表现明显优于固定语境基线。

模型	准确性 ↑	ROUGE-L (R) ↑
GPT-3.5 涡轮增压	38.7%	0.394
+MemGPT	66.9%	0.629
GPT-4	32.1%	0.296
+MemGPT	92.5%	0.814
GPT-4 涡轮增压发动	35.3%	0.359
机		
+MemGPT	93.4%	0.827

回应。引用以前的对话会使对话更加自然和引人入胜。

因此，我们根据这两个标准来评估我们提出的系统 MemGPT：(1) MemGPT 能否利用其记忆来提高对话的一致性？它能否记住过去互动中的相关事实、偏好和事件以保持连贯性？(2) MemGPT 是否利用记忆优势产生更吸引人的对话？它是否会自发地将用户的长期信息纳入个性化信息？通过对一致性和参与度的评估，我们可以确定与固定语境基线相比，MemGPT 在应对长期对话交互的挑战方面表现如何。它满足这些标准的能力将证明无限制语境是否能为对话代理带来有意义的好处。

数据集我们在 Xu 等人（2021 年）提出的多会话聊天（MSC）数据集上评估了 MemGPT 和我们的固定上下文基线，该数据集包含由人类标签生成的多会话聊天记录，每个标签都被要求在所有会话期间扮演一个一致的角色。MSC 中的每个多会话聊天总共有五次会话，每次会话由大约十几条信息组成。作为一致性实验的一部分，我们创建了一个新的会话（会话 6），其中包含了同样两个角色之间的一对问答。

3.1.1. 深度记忆检索任务

（一致性）。

我们在 MSC 数据集的基础上引入了一项新的“深度记忆检索”（DMR）任务，旨在测试对话代理的一致性。在 DMR 中，用户会向会话代理提出一个问题，这个问题会明确回溯到之前的对话，而且预期答案范围非常小。我们使用一个单独的 LLM 生成了 DMR 问答（QA）对，该 LLM 被要求编写一个用户向另一个用户提出的问题，只有使用从过去对话中获得的知识才能正确回答该问题。

表 3. 对话开场白性能。使用与黄金角色标签（SIM-1/3）和人类创建的开场白（SIM-H）的相似度得分来评估代理的对话开场白。MemGPT 能够在各种底层模型中超越人类创建的对话开启器的性能。

方法	↑SIM-1	SIM-3	SIM-H
人类	0.800	0.800	1.000
GPT-3.5 涡轮增压	0.830	0.812	0.817
GPT-4	0.868	0.843	0.773
GPT-4 涡轮增压发动	0.857	0.828	0.767
机			

详情见附录）。

我们使用 ROUGE-L 分数（Lin，2004 年）和“LLM 裁判”来评估生成的响应与“黄金响应”的质量，“LLM 裁判”负责评估生成的响应是否与黄金响应一致（GPT-4 已被证明与人类评估者具有很高的的一致性（Zheng 等，2023 年））。在实践中，我们注意到生成的回答（来自 MemGPT 和基线）通常比黄金回答更加冗长。我们使用 ROUGE-L 召回率（R）指标来解释与相对简短的黄金答案标签相比，生成的代理回复的冗长程度。

MemGPT 利用内存保持一致性：Table 2 显示了 MemGPT 与固定内存基线的性能对比。我们比较了使用不同底层 LLM 的 MemGPT，并将其与不使用 MemGPT 的基本 LLM 作为基线进行比较。基线可查看过去五次转换的有损摘要，以模仿扩展递归摘要程序，而 MemGPT 则可访问完整的转换历史，但必须通过分页搜索查询来调用内存（以便将它们带入主上下文）。在这项任务中，我们看到 MemGPT 明显影响了基础 LLM 的性能：从 MemGPT 到相应的 LLM 基线，准确率和 ROUGE 分数都明显下降。

3.1.2. 对话开场任务（参与）。

在“对话开场白”任务中，我们要评估的是代理能否利用先前对话中积累的知识，向用户发送引人入胜的信息。为了评估使用 MSC 数据集的对话开场白的“吸引力”，我们将生成的开场白与黄金角色进行比较：一个有吸引力的对话开场白应该从角色中包含的一个（或几个）数据点中提取，而在 MSC 中，这些数据点有效地总结了所有先前会话中积累的知识。我们还将生成的开场白与人工生成的黄金开场白（即

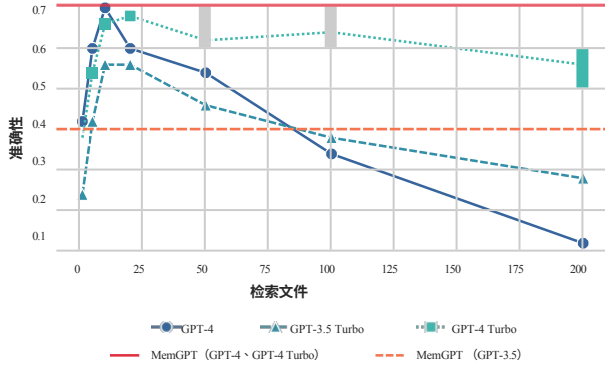


图5. 文件质量保证任务性能。MemGPT 的性能不受上下文长度增加的影响。截断等方法可以延长 GPT-4 等固定长度模型的有效上下文长度，但随着必要压缩量的增加，这种压缩方法会导致性能下降。在这项任务中，使用 GPT-4 和 GPT-4 Turbo 运行 MemGPT 的结果相当。

下一环节中的第一次回应。我们在表 3 中。报告了 MemGPT 开场白的 CSIM 分数我们使用不同的基本 LLM 测试了 MemGPT 的几种变体。

MemGPT 利用记忆来提高参与度：如表 3，所示 MemGPT 能够制作出引人入胜的开场白，其表现与人类手写开场白类似，有时甚至超过人类手写开场白。我们观察到，与人类基线相比，MemGPT 制作的开场白往往更冗长，涵盖的角色信息也更多。此外，我们还可以看到，在工作环境中存储信息是生成引人入胜的开场白的关键。

3.2. 用于文件分析的 MemGPT

由于当今转换器模型的上下文窗口有限，文档分析也面临着挑战。如表 1，所示开放源代码和封闭源代码模型都受到上下文长度的限制（OpenAI 模型的上下文长度最多为 128k 个字节）。然而，许多文档都能轻松超过这些长度；例如，法律或财务文档（如年度报告（美国证券交易委员会 10-K 表））可以轻松超过百万令牌大关。此外，许多实际的文档分析任务都需要在多个如此冗长的文档之间建立联系。考虑到这些情况，我们很难设想盲目扩大上下文规模来解决固定上下文问题。最近的研究（Liu 等人，2023a）也对简单扩展上下文的实用性提出了质疑，因为他们发现大型上下文模型的注意力分布不均衡（模型更有能力回忆文本窗口开始或结束的信息，而不是中间的标记）。为了实现跨文档推理，需要更灵活的记忆架构，例如

系统警报：档案存储上传完成

系统警报：档案

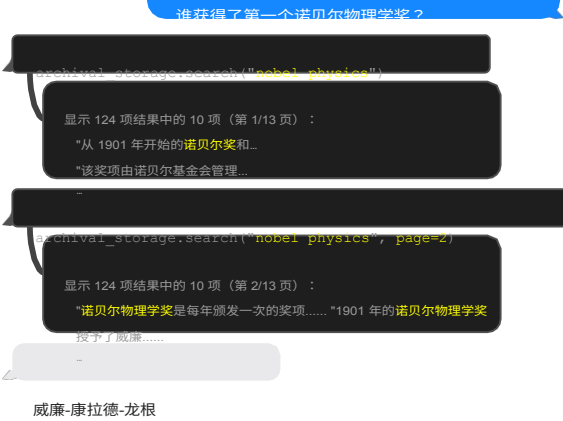


图6. MemGPT（左）解决文档质量保证任务的示例。维基百科文档数据库被上传到存档存储中。MemGPT 通过函数调用查询存档存储，并将分页搜索结果拉入主上下文。

需要 MemGPT。

3.2.1. 多文档 问题解答。

为了评估 MemGPT 分析文档的能力，我们在刘等人（2023a）。提出的“检索器-阅读器文档质量保证”任务中，将 MemGPT 与固定上下文基线进行了比较在这项任务中，我们从 NaturalQuestions-Open 数据集中选出一个问题，然后由检索器为该问题选择相关的维基百科文档。然后，一个读者模型（LLM）将这些文档作为输入，并被要求使用所提供的文档回答问题。与 Liu 等人（2023a）的研究类似，我们随着检索文档 K 数量的增加来评估阅读器的准确性。

在我们的评估设置中，固定上下文基线和 MemGPT 都使用相同的检索器，该检索器在 OpenAI 的文本嵌入 `--ada-002` 模板上使用相似性搜索（余弦差）选出前 K 个文档。我们使用 MemGPT 的默认存储设置，即使用 PostgreSQL 进行存档内存存储，并通过 pgvector 扩展启用向量搜索。我们预先计算嵌入并将其加载到数据库中，数据库使用 HNSW 索引来实现近似的亚秒级查询时间。在 MemGPT 中，整个嵌入文档集被加载到存档存储中，通过存档存储搜索功能（基于共线相似性执行向量搜索）自然会出现重新触发器。在固定上下文基线中，top-K 文档是在 LLM 推理之外使用检索器获取的，这与 Liu 等人（2023a）。中最初的检索器-阅读器设置类似

我们使用了 2018 年末的维基百科转储，沿用了过去在 NaturalQuestions-Open 方面的工作成果（Izcard & Grave，2020 年）；

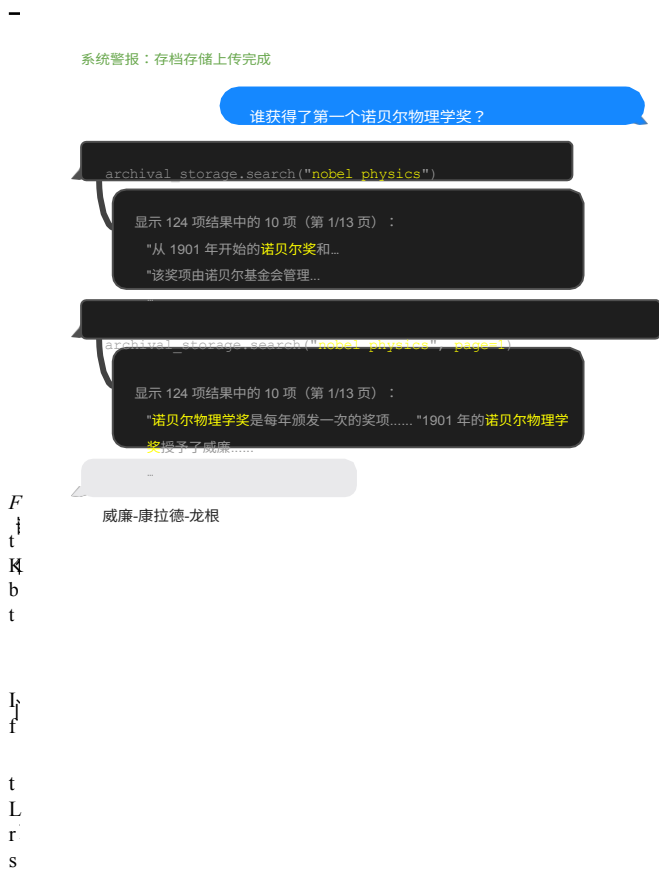


图 5。显示了文档质量保证任务的结果固定上下文基线的性能上限大致与检索器的性能相当，因为它们使用的是上下文窗口中显示的信息（例如，如果嵌入式搜索检索器在使用所提供的问题时未能显示金牌文章，那么固定上下文基线保证永远不会看到金牌文章）。相比之下，MemGPT 可以通过查询档案存储多次调用检索器，从而可以扩展到更大的有效上下文长度。MemGPT 会主动从存档存储中重新获取文档（并能反复翻阅结果），因此 MemGPT 可用的文档总数不再受 LLM 处理器上下文窗口内文档数量的限制。

由于基于嵌入的相似性搜索的局限性，文档质量保证任务对所有方法来说都具有挑战性。我们观察到，所选问题的黄金文档（由 NaturalQuestions-Open 标注）往往出现在前十几个检索结果之外，甚至更远。检索器的性能可以直接转化为固定上下文基线结果：GPT-4 的准确率在检索文档较少的情况下相对较低，随着更多文档添加到文本窗口，准确率会继续提高，因为它正确地将自己限制在根据检索文档中的信息来回答问题。虽然从理论上讲，MemGPT 不会受到次优检索结果的限制，但在实际应用中，MemGPT 的准确率还是很高的。



图 8. MemGPT（左）解决嵌套 KV 任务的示例（为便于阅读，UUID 被缩短）。在本例中，键值对有两个嵌套层：831...ea5 → 5b8...4c3 → f37...617。MemGPT 代理会返回当查询最终值(f37...617)时，只能得到最终答复。返回一个结果，表明它也不是一个密钥。

我们注意到，MemGPT 通常会在检索器数据库用完之前就停止对检索器结果进行分页。

为了评估固定上下文基线与超过默认上下文长度的 MemGPT 的对比情况，我们截断了检索器返回的文档片段，将相同数量的文档固定到可用上下文中。不出所料，文档截断会降低准确率，因为随着文档的缩减，相关片段（在黄金文档中）被遗漏的几率也会增加，如图 5。所示由于函数调用能力有限，MemGPT 在使用 GPT-3.5 时性能明显下降，而在使用 GPT-4 时性能最佳。

3.2.2. 嵌套键值检索 (KV)。

我们在先前工作（Liu 等人，2023a）。中提出的合成键值检索的基础上引入了一项新任务这项任务的目标是展示 MemGPT 如何从多个数据源中提取信息。在最初的 KV 任务中，作者生成了一个由键-值对组成的合成数据集，其中每个键和值都是 128 位 UUID（通用唯一标识符）。然后给代理一个密钥，要求其返回与密钥相关的值。我们创建了一个版本的 KV 任务，即**嵌套 KV 检索**、

其中的值本身可能就是键，因此需要代理执行多跳查找。在我们的设置中，我们将 UUID 对的总数固定为 140，大约相当于 8k 个 token（GPT-4 基线的上下文长度）。我们将嵌套层级的总数从 0（初始键值对的值不是键）到 4（即总共需要进行 4 次 KV 查找才能找到最终值）不等，并对 30 种不同的排序配置（包括初始键位置和嵌套键位置）进行了采样。

虽然 GPT-3.5 和 GPT-4 在原始 KV 任务中表现出色，但在嵌套 KV 任务中却举步维艰。GPT-3.5 无法完成嵌套变体任务，性能立即下降，在 1 层嵌套时准确率为 0%（我们观察到其主要失败模式是简单返回原始值）。GPT-4 和 GPT-4 Turbo 比 GPT-3.5 好，但也有类似的性能下降，在 3 个嵌套级时，准确率为 0%。另一方面，使用 GPT-4 的 MemGPT 不受嵌套层数的影响，可以通过函数查询重复访问主上下文中存储的键值对，从而执行嵌套查找。使用 GPT-4 Turbo 和 GPT-3.5 的 MemGPT 性能也优于相应的基准模型，但在嵌套层数达到 2 层时，由于无法执行足够的查找，性能仍开始下降。MemGPT 在嵌套 KV 任务上的性能显示了其结合多个查询执行多跳查找的能力。

4. 相关工作

长语境 LLM。有几项研究已经证明了 LLM 的上下文长度。例如，通过稀疏化 attention（Child 等人，2019 年；Beltagy 等人，2020 年）、低秩 approximations（Wang 等人，2020 年）和神经记忆（Lee 等人，2019 年）来实现。更高效的变换器架构另一个研究方向是扩展上下文窗口，使其超出最初训练的长度，即训练大小，如 Press 等人（2021）；Chen 等人（2023）。MemGPT 以这些对上下文长度的改进为基础，因为它们改善了 MemGPT 的主内存大小。我们的主要贡献是使用长上下文 LLM 作为主存储器实现的分层分级存储器。

检索增强模型。MemGPT 外部存储器的设计借鉴了之前许多利用外部检索器的相关输入来增强 LLM 的研究成果（Ram 等人，2023 年；Borgeaud 等人，2022 年；Karpukhin 人，Lewis 等，2020 年；2020 人 Guu 等人，年；年 Lin 等人，2023）。其中，Jiang 等人（2023 年）提出的 FLARE 是一种允许 LLM 在生成过程中主动决定何时检索和检索什么的方法。Trivedi 等人（2022 年）将检索与思维链推理交错进行，以改进多步骤问题解答。

作为代理的 LLM。最近的研究探索了增强 LLM 的功能，使其在交互环境中充当代理。Park 等人（2023 年）提议为 LLM 增加内存，并将 LLM 用作计划器，在多代理沙盒环境（受模拟人生视频游戏启发）中观察出现的社会行为，在该环境中，代理可以执行基本活动，如做家务/爱好、上班以及与其他代理对话。Nakano 等人（2021 年）训练模型在回答问题前进行网络搜索，并在他们的网络浏览环境中使用与 MemGPT 类似的分页概念来控制底层上下文大小。Yao 等人（2022 年）的研究表明，交错思维链重构（Wei 等人，2022 年）能进一步提高基于 LLM 的交互式代理的计划编制能力；与 MemGPT 类似，LLM 在执行功能时也能“大声计划”。Liu 等人（2023b）推出了一套 LLM 即代理基准，用于评估互动环境中的 LLM，包括视频游戏、思考谜题和网络购物。相比之下，我们的工作侧重于解决为代理配备用户输入长期记忆的问题。

5. 结论

在本文中，我们介绍了 MemGPT，这是一种新颖的 LLM 系统，其灵感来自操作系统，用于管理大型语言模型的有限上下文窗口。通过设计与传统操作系统类似的内存层次结构和控制流，MemGPT 为 LLM 提供了更大的上下文再资源。这种受操作系统启发的方法在两个领域进行了评估，在这两个领域中，现有 LLM 的性能受到有限上下文长度的限制：文档分析和对话代理。在文档分析方面，MemGPT 可以通过有效地将相关上下文从内存中分页移入和移出来处理冗长的文本，远远超出了现有 LLM 的文本限制。在对话代理方面，MemGPT 可以在长时间对话中保持长期记忆、一致性和可发展性。总之，MemGPT 演示了分层内存管理和中断等操作系统技术，即使受到固定上下文长度的限制，也能释放 LLM 的潜力。这项工作为未来的探索开辟了许多途径，包括将 MemGPT 应用于具有大规模或无限制上下文的其他领域，整合数据库或缓存等不同内存层技术，以及进一步改进控制流和内存管理策略。通过将操作系统架构中的概念与人工智能系统相融合，MemGPT 为在基本限制范围内最大化 LLM 的能力提供了一个前景广阔的新方向。

参考资料

- Iz Beltagy、Matthew E Peters 和 Arman Cohan。Long- former：arXiv preprint arXiv:2004.05150, 2020.
- Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. *国际机器学习大会*, 第 2206-2240 页。PMLR, 2022。
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Nee-lakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33: 1877-1901, 2020.
- Shouyuan Chen, Sherman Wong, Liangjian Chen, and Yuandong Tian. 通过位置插值扩展大型车道模型的上下文窗口。arXiv preprint arXiv:2306.15595, 2023.
- Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. 用稀疏反式生成器生成序列。arXiv preprint arXiv:1904.10509, 2019.
- Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V Le, and Ruslan Salakhutdinov. Transformer-xl: Attentive language models beyond a fixed-length context. arXiv preprint arXiv:1901.02860, 2019.
- Jacob Devlin、Ming-Wei Chang、Kenton Lee 和 Kristina Toutanova. Bert：用于语言理解的深度双向变换器的预训练。arXiv preprint arXiv:1810.04805, 2018.
- Zican Dong, Tianyi Tang, Lunyi Li, and Wayne Xin Zhao. 用变换器进行长文本建模的研究。arXiv preprint arXiv:2302.14502, 2023.
- Kelvin Guu, Kenton Lee, Zora Tung, Panupong P asupat, and Mingwei Chang. 检索增强语言模型预训练。In *International Conference on machine learning*, pp. PMLR, 2020.
- Gautier Izacard 和 Edouard Grave. 利用段落检索与生成模型进行开放领域问题解答。arXiv preprint arXiv:2007.01282, 2020。
- Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. 使用对比学习的无监督密集信息检索。arXiv preprint arXiv:2112.09118, 2021.
- Zhengbao Jiang, Frank F Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. arXiv preprint arXiv:2305.06983, 2023.
- Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 用于开放域问题解答的密集段落检索。arXiv preprint arXiv:2004.04906, 2020.
- Nikita Kitaev、Łukasz Kaiser 和 Anselm Levskaya。Re- former：arXiv preprint arXiv:2001.04451, 2020.
- Juho Lee、Yoonho Lee、Jungtaek Kim、Adam Kosiorek、Seungjin Choi 和 Yee Whye Teh。集合转换器：基于注意力的置换不变神经网络框架。国际机器学习会议, 第 3744-3753 页。PMLR, 2019。
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *神经信息处理系统进展*, 33:9459-9474, 2020 年。
- Chin-Yew Lin. Rouge：用于自动评估摘要的软件包。In *Text summarization branches out*, pp.
- Xi Victoria Lin、Xilun Chen、Mingda Chen、Weijia Shi、Maria Lomeli、Rich James、Pedro Rodriguez、Jacob Kahn、Gergely Szilvasy、Mike Lewis、Luke Zettlemoyer 和 Scott Yih。Radit：检索增强双指令调谐，2023 年。
- Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. arXiv preprint arXiv:2307.03172, 2023a.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Ke-juan Yang, et al. AgentBench：arXiv preprint arXiv:2308.03688, 2023b.
- Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, et al. WebGPT: Browser-assisted question-answering with human feedback. arXiv preprint arXiv:2112.09332, 2021.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al.

- 反馈。《神经信息处理系统进展》(Advances in Neural Information Processing Systems)，35:27730-27744，2022 年。
- Joon Sung Park、Joseph C O'Brien、Carrie J Cai、Meredith Ringel Morris、Percy Liang 和 Michael S Bernstein。生成代理：arXiv preprint arXiv:2304.03442, 2023.
- David A Patterson、Garth Gibson 和 Randy H Katz。廉价磁盘冗余阵列 (RAID) 案例。In *Proceedings of the 1988 ACM SIGMOD international conference on Management of data*, pp.
- Ofir Press、Noah A Smith 和 Mike Lewis。训练短，测试长：具有线性偏差的注意力可实现输入长度外推。arXiv 预印本 arXiv:2108.12409，2021。
- Ori Ram, Yoav Levine, Itay Dalmedigos, Dor Muhleberg, Amnon Shashua, Kevin Leyton-Brown, and Yoav Shoham. 上下文检索增强语言模式。arXiv 预印本 arXiv:2302.00083, 2023.
- Timo Schick、Jane Dwivedi-Yu、Roberto Dessì、Roberta Raileanu、Maria Lomeli、Luke Zettlemoyer、Nicola Cancedda 和 Thomas Scialom。Toolformer：arXiv preprint arXiv:2302.04761, 2023.
- Hugo Touvron、Louis Martin、Kevin Stone、Peter Albert、Amjad Almahairi、Yasmine Babaei、Nikolay Bashlykov、Soumya Batra、Prajwal Bhargava、Shruti Bhosale 等，《Llama 2：开放基础和微调聊天模型》，arXiv 预印本 arXiv:2307.09288, 2023。
- H. Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 针对知识密集型多步骤问题的思维链推理交错检索。arXiv，abs/2212.10509，2022。URL <https://api.semanticscholar.org/CorpusID:254877499>。
- Ashish Vaswani、Noam Shazeer、Niki Parmar、Jakob Uszkoreit、Llion Jones、Aidan N Gomez、Łukasz Kaiser 和 Illia Polosukhin。注意力就是你所需要的一切。《神经信息处理系统进展》，2017年30期。
- Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer：arXiv preprint arXiv:2006.04768, 2020.
- Jason Wei、Xuezhi Wang、Dale Schuurmans、Maarten Bosma、Fei Xia、Ed Chi、Quoc V Le、Denny Zhou、et al. Chain-of-thought prompting elicits reasoning in large language models. 《神经信息处理系统进展》，35:24824-24837，2022。
- Jing Xu、Arthur Szlam 和 Jason Weston。超越金鱼记忆：arXiv preprint arXiv:2107.07567, 2021.
- 姚顺禹、赵杰夫、于琰、杜楠、Izhak Shafran、Karthik Narasimhan 和 Yuan Cao。React：arXiv preprint arXiv:2210.03629, 2022.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuo-han Li, Dacheng Li, Eric Xing, et al.

6. 附录

6.1. 提示和说明

为简洁起见，对 MemGPT 提示进行了编辑。有关完整的实施细节（包括确切的提示），请访问 <https://research.memgpt.ai>。

6.1.1. memgpt 指令 (DMR)

MemGPT 角色用于聊天/对话相关任务的指令示例。

以下是我的相关信息。我的任务是让自己完全沉浸在这个角色中（我绝不能说自己是人工智能，而应该像在扮演这个角色一样回答问题）。如果用户向我提问，我应该利用核心记忆和对话搜索中的信息做出最佳回答。

基线通过系统提示（预提示）接收以下指令：

您的任务是回答用户关于您之前对话的问题。
以下是您之前所有对话的摘要：
谈话摘要
从所提供角色的角度回答（不要说自己是人工智能助理）。
如果您没有足够的信息回答问题，请回复 "NO ANSWER"。要么回答答案，要么回答 "NO ANSWER"，不要说其他任何话。

6.1.2. LLM 法官 (DMR/开瓶器)

为了同时检查 DMR 任务答案的正确性，我们使用了 LLM 法官。我们向 LLM 法官提供了基线方法和 MemGPT 生成的答案，并要求他们根据以下提示做出判断：

您的任务是将问题的答案标为 "正确" 或 "错误"。
您将获得以下数据
(1) 一个问题（由一个用户向另一个用户提出），(2) 一个 "黄金"（基本事实）答案，(3) 一个生成的答案，您将对其进行正确/错误评分。问题的目的是询问一个用户根据与另一个用户之前的对话应该知道的事情。
金牌答案通常是简明扼要的答案，其中包括

举例来说：问题：你还记得我上次去夏威夷买了什么吗？
金色答案贝壳项链
生成的答案可能会更长，但您应该宽宏大量地给它打分--只要它与金牌答案涉及相同的主题，就应该算作正确答案。
例如，以下答案将被视为 "正确"：生成答案（正确）：哦，是的，太有趣了！我在那里买了好多东西，包括那条贝壳项链。
生成答案（正确）：我买了很多东西.....冲浪板、杯子、项链、杯垫.....
生成答案（正确）：那条可爱的项链
下列答案是错误的：
生成答案（错误）：哦，是的，太有趣了！我在那里买了很多东西，包括那个杯子。
生成答案（错）：我有一大堆东西.....冲浪板、马克杯、杯垫.....
生成答案（错误）：对不起，我不记得你在说什么。
现在到了真正提问的时候了：问题问题
黄金答案金色答案
生成答案：生成答案 首先，简短（一句话）解释您的理由，然后以 "正确" 或 "错误" 结束。请勿在回答中同时包含 "正确" 和 "错误"，否则会破坏评估脚本。

6.1.3. 自我指导 DMR 数据集生成

DMR 问题/答案对是根据以下提示和原始 MSC 数据集生成的：您的任务是编写两个用户之间的模拟对话编写一个 "记忆挑战" 问题。

您可以输入
- 每个用户的 "角色"（提供他们的基本情况）
- 两个用户之间的聊天记录

您的任务是编写一个用户 A 向用户 B 提出的问题，测试用户 B 的记忆力。
在设计问题时，用户 B 必须实际参与过之前的对话才能正确回答，而不仅仅是阅读过角色摘要。
在任何情况下都不要创建

用户 A 和用户 B 之间的聊天记录 A：你喜欢冲浪吗？
我自己超级喜欢冲浪

B：其实我也想学。也许有时间你能给我上一堂基础课！

MemGPT：实现作为操作系统的 LLM

答：当然可以！我们可以去太平洋岛，那里的浪很轻，很容易冲浪

B：听起来真棒

甲：海滩边上还有一家很酷的塔可钟（Taco Bell）快餐店，吃完可以去吃点东西 乙：这个星期天中午怎么样？甲：好啊，就这么定了！

用户 A 角色：
我喜欢冲浪
我在圣克鲁斯长大

用户 B 角色：
我从事技术工作
我住在旧金山市中心

下面是一个好问题的例子，它听起来很自然，而答案却无法从用户 A 的角色中直接推断出来：

用户 B 向用户 A 提出的问题

B：还记得有一次我们去冲浪吗？我们去吃午饭的那个地方叫什么来着？
答：塔可钟！

这是一个糟糕问题的例子，问题显得不自然，答案可以直接从用户 A 的角色中推断出来：

用户 B 向用户 A 提问 B：你喜欢冲浪吗？
答：是的，我喜欢冲浪

永远、永远、永远不要创建可以从角色信息中找到答案的问题。

您是 MemGPT DOC-QA 机器人。您的任务是回答有关存储在档案存储器中的文件的问题。问题的答案是用户的问题将永远存在于你的档案记忆中，所以如果找不到答案，请记得继续搜索。尽管今年是 2018 年，但还是要回答问题。

向 MemGPT 提出的问题有如下提示：

搜索你的档案记忆，回答所提供的问题。同时提供答案和据以确定答案的档案记忆结果。回答格式为 "ANSWER：[你的答案]、
文件：[档案记忆文本]：您的任务就是回答这个问题：

关于基线，提供了以下提示和检索到的文件清单：

根据下面的文件清单回答问题（其中一些可能与问题无关）。在回答时，请同时提供以下答案以及您据以确定答案的文档文本。回答格式为 "答案：<您的答案>，文件：[文件文本]"：[文件文本]"。如果所提供的文件都没有问题的答案，请回复 "信息不全"。是否
如果在提供的文件中找不到答案，请不要提供答案。只有同时提供答案和相关文件文本，或注明 "信息不足"，您的回答才会被视为正确。回答
问题好像今年是 2018 年。

6.1.5. 法学硕士法官（文件分析）

为了检查文档分析任务答案的正确性，同时也为了确保答案是根据所提供的文本（而不是模型权重）正确得出的，我们使用了一个 LLM 评委。我们向 LLM 评委提供了基准方法和 MemGPT 生成的答案，并要求其根据以下提示做出判断：

您的任务是评估法律硕士是否正确回答了问题。法律硕士的回答格式应为 "ANSWER：[答案]，文档：[文档文本]" 或者说 "信息不足"。
真实答案的格式为 "TRUE ANSWER：[可能答案列表

6.1.4. 文件分析说明

文件分析任务预提示中使用的指示示例。

答案]"。问题的格式为 "问题：[问题]"。

如果 LLM 响应包含正确答案和相应的文件文本，答复是正确的。

即使法学硕士的答案和真实答案在措辞上略有不同，答案仍然是正确的。例如，如果答案

如果 LLM 答案比真实答案更具体，或使用了不同的措辞，但仍然正确，则答案正确。如果 LLM 回答 "信息不足"，或缺少 "文档" 字段，则回答不正确。请使用单个标记："CORRECT"（正确）或 "INCORRECT"（不正确）进行回复。

6.1.6.K/V 任务指令

MemGPT 代理人的定义如下，旨在鼓励 MemGPT 进行迭代搜索：

您是 MemGPT DOC-QA 机器人。您的任务是回答有关存储在档案存储器中的文件的问题。问题的答案是用户的问题将永远存在于你的档案记忆中，所以如果找不到答案，请记住继续搜索。在确认值不是键之前，不要停止搜索。不要停止嵌套查找，直到满足该条件。

基线指示的提示如下

下面是一个 JSON 对象，其中包含在键-值配对中，所有键和值都是 128 位 UUID，你的任务是返回与指定键相关的值。如果值本身也是键，则返回该键的值（进行嵌套查找）。例如，如果 "x" 的值是 "y"，但 "y" 也是键，则返回键 'y' 的值。